



Chapter 11 — Testing, Risks & Success Metrics

11.1 Test Strategy Overview

DOIP follows a **test pyramid adapted for a real-time, AI-heavy system**: fast deterministic unit tests at the base, integration tests around the event pipeline, a thin layer of end-to-end UI tests, and a dedicated evaluation harness for AI quality (which is measured, not asserted).

Layer	Tooling	Scope	Runs
Unit	pytest, vitest	Domain logic, cost functions, DSL, reducers	Every push
Integration	pytest + Compose services	API + DB + Redis + WS end-to-end paths	Every push
Golden-run (determinism)	pytest	Seeded scenario → exact event-sequence hash	Every push
E2E	Playwright	Login → start run → map updates → incident flow	Main branch
Load	Locust / k6	200 entities, 50 events/s, 20 WS clients	Per phase
AI evaluation	custom harness (<code>make eval-ai</code>)	Sitrep grounding, RAG retrieval, CV mAP	Nightly

11.2 What Each Layer Must Cover

Unit tests (fast, no I/O)

- Route cost function: weather/terrain/restricted-zone math; A* returns optimal path on known grids; Dijkstra parity check.
- Scenario DSL: every condition/action function; malformed YAML rejected with line numbers.
- Battery/consumption models: conservation properties, never-negative invariants.
- Risk scoring formula: class weights, clustering, proximity factor edge cases.

- Frontend reducers: every event type in the registry has a handled case (exhaustiveness-checked in TypeScript).

Integration tests

- Event round-trip: engine tick → DB row → bus → WS client receives within budget.
- WS resume: kill connection mid-run → reconnect with `last_event_id` → zero missed discrete events.
- Snapshot consistency: snapshot + subsequent events == replayed full stream.
- RBAC: every endpoint tested against all four roles (authorization matrix as parameterized test).
- Crash recovery: SIGKILL the engine mid-run → restart → world state matches projections; sim resumes.

Golden-run determinism suite (the platform's signature test)

- Scenario seed 42, 500 ticks → SHA-256 of the ordered event stream must match the committed fixture.
- Runs at 1x and 20x speed → identical hashes (speed invariance).
- Any intentional engine change requires regenerating the fixture in the same PR — making behavioral changes visible in review.

AI evaluation harness (quality, not pass/fail)

Metric	Target	Regression gate
Sitrep grounding rate (claims with valid event citations)	≥ 98%	fail < 95%
Sitrep hallucination count on golden set	0	fail > 0
RAG retrieval hit-rate (correct source in top-4)	≥ 85%	fail < 75%
RAG citation precision	≥ 90%	fail < 80%
CV mAP@50 on validation clip (pretrained classes)	≥ 0.45	fail < 0.35
Sitrep p95 latency	< 15 s	fail > 20 s

11.3 Risk Register

#	Risk	Likelihood	Impact	Mitigation
R1	Scope explosion — trying to build all 10 modules at once	High	Critical	Phase gates with exit criteria (Ch. 9); cut-list §9.10; never start phase N+1 early

#	Risk	Likelihood	Impact	Mitigation
R2	Simulation engine under-designed, downstream modules build on sand	Medium	Critical	Engine designed first (Ch. 7); golden-run suite from Phase 1; ADR-6 single-writer rule
R3	Local LLM quality too low for convincing sitreps	Medium	High	Grounding verification strips bad claims; prompt iteration on golden set; provider interface allows hosted-API swap for demos
R4	WebSocket performance issues with many entities	Medium	Medium	Batched 4 Hz position frames; lossy-positions/lossless-events split; load test each phase
R5	CV accuracy poor on smoke/fire/damage classes	Medium	Medium	Ship pretrained vehicle/person first; fine-tune only if time; reviewer UI makes false positives harmless
R6	Kafka/K8s migration destabilizes a working system	Medium	High	Migration is last (Phase 7); dual-write cutover; Compose remains a supported fallback
R7	Solo-developer burnout / momentum loss	Medium	High	Every phase ends with a visible demo; timeboxing; the cut-list keeps the finish line real
R8	Hardware limits (no GPU) slow AI/CV development	Medium	Medium	Quantized 7B models; YOLO nano; 2 fps frame sampling; all budgets sized for CPU in Ch. 4 §4.6

#	Risk	Likelihood	Impact	Mitigation
R9	Misperception that the project controls real systems	Low	High	Simulation-only banner in UI, README, and docs; disaster-response scenario framing; non-goals in Ch. 1 §1.8
R10	Map tile service rate limits	Low	Low	OpenFreeMap/self-hosted tiles fallback; tile cache in Nginx

11.4 Success Metrics (KPIs)

Platform health (measured continuously in Grafana)

- Event latency simulator → browser: **p95 < 1 s** (NFR-1)
- Sustained throughput: **200 entities, 50 events/s** without tick overrun (NFR-2)
- Golden-run suite: **100% deterministic** across machines
- Zero lost discrete events across reconnect tests

AI quality (nightly eval trend)

- Grounding ≥ 98%, hallucinations = 0, RAG hit-rate ≥ 85% (see §11.2 targets)

Project delivery

- 7 tagged releases, each with demo recording + changelog
- README quickstart verified: clone → running stack < 10 min
- Test coverage: ≥ 80% on engine + mission domains; every event type integration-tested

Showcase impact

- The 3-minute demo runs end-to-end without manual intervention, on one laptop, offline.

11.5 The 3-Minute Demo Script (the artifact everything optimizes for)

1. **0:00** — Login as operator; dashboard loads with the seeded “Flood Response” scenario ready.

2. **0:15** — Start run at 5x. Units begin patrolling; event feed streams; map markers glide.
3. **0:45** — Rain cell spawns and drifts over Sector B (overlay visible); Team Bravo visibly slows; speed-factor tooltip shows why.
4. **1:15** — Critical medical incident fires; map pings; “units within 5 km” panel ranks responders by distance.
5. **1:30** — Click **Generate Sitrep**: AI produces a prioritized summary citing the actual events; open one citation.
6. **2:00** — Ask the assistant: *“What’s the SOP for medical evacuation in heavy rain?”* → cited RAG answer from the SOP corpus.
7. **2:20** — Plan a drone survey around the storm: draw zone → optimize → route avoids the restricted zone and rain cell → launch → watch it fly.
8. **2:45** — Logistics panel: medkit forecast dips toward threshold with uncertainty band; early-warning alert appears.
9. **3:00** — Stop run → after-action report renders: timeline, response time, resource curves, AI narrative. *End*.

11.6 After-Action Review Metrics (in-product analytics)

For every completed run, the platform itself reports:

- Mean/percentile incident response time (creation → acknowledgment → resolution)
- Mission success rate and ETA forecast error (predicted vs actual)
- Resource efficiency: consumed vs forecast; shortage warnings lead time
- Alert precision: confirmed vs dismissed detections and health alerts
- Operator activity timeline for training review

These numbers make DOIP genuinely useful as a *training* platform — trainees can be debriefed against objective metrics, and repeated runs of the same seeded scenario are directly comparable.